

Coding for Artificial Intelligence

From Script to Package

Problem Set 8

Packaging a Module and Automated Style Linting

Project: Packaging & Refactoring the Local AI Text Utility

Course	Coding for AI
Author	Dr. Prateek, Assistant Professor, School of AI
Format	In-class project, 1 hour 30 minutes total, inclusive of write-up and submission
Total Marks	100

Read this document in full before you begin writing code.

Contents

1	Objective	2
2	Task 1: Creating the Distribution Package (35 minutes)	2
2.1	Directory Setup	2
2.2	Writing <code>setup.py</code>	2
2.3	Building and Installing	3
3	Task 2: Diagnostic Linting and PEP 8 Refactoring (40 minutes)	3
3.1	Install the Checker	3
3.2	Intentional Flaw Injection	3
3.3	Running the Linter	3
3.4	Reading and Fixing the Output	3
3.5	Verification	4
4	Edge Cases and Extensions	4
5	Suggested Time Allocation	5
6	Deliverables	5
	Academic Integrity	5

1 Objective

Every function you have written so far has lived inside a single script, run once and discarded. This project takes your `search4letters` function (as discussed during class lecture) and turns it into something more durable: a **locally installable Python package** that any script on your machine can `import`, and then puts that package through an **automated style checker** to practise reading and acting on real diagnostic tool output.

By the end of this project you should be able to:

- (i) structure a folder and a `setup.py` file so that a module becomes an installable package,
- (ii) build, install, and verify a package using standard command-line tools, and
- (iii) read the output of a PEP 8 style checker and correct each class of violation it reports.

Time budget. This entire activity — coding, testing, and writing up your submission — is designed to fit in **90 minutes**. A suggested breakdown is given in Section 5.

A note on tooling. This lab uses `pycodestyle` (the actively maintained successor to the original `pep8` tool) and `python -m build` (the current standard packaging front-end). Older material you may find online references `py.test --pep8` and `python setup.py sdist` directly — the `pytest-pep8` plugin those commands depend on has not been updated in years and **no longer works** with current versions of `pytest`, and direct `setup.py` invocation is considered legacy practice by the Python Packaging Authority. The underlying `setup.py` file you write in Task 1 is still exactly right; only the *commands* used to build and check the package have been updated to their modern equivalents.

2 Task 1: Creating the Distribution Package (35 minutes)

2.1 Directory Setup

Create a folder named `text_utility_pkg` containing:

- (i) `vsearch.py`, containing your `search4letters` function as discussed earlier.
- (ii) An empty `README.txt` file.

2.2 Writing `setup.py`

Create `setup.py` inside `text_utility_pkg` with the following configuration:

```
1 from setuptools import setup
2
3 setup(
4     name='vsearch',
5     version='1.0',
6     description='Text search tools for AI pre-processing',
7     author='Mtech AI Student',
8     author_email='student@school.edu',
9     url='localhost',
10    py_modules=['vsearch'],
11 )
```

2.3 Building and Installing

From inside `text_utility_pkg`, run the following commands [applicable to Linux; for Windows, use `python`].

```
python3 -m pip install --upgrade build
python3 -m build --sdist
```

This produces `dist/vsearch-1.0.tar.gz`. Now install it:

```
python3 -m pip install dist/vsearch-1.0.tar.gz
```

Verified. These exact commands were run against the `setup.py` above and produced `dist/vsearch-1.0.tar.gz` without error, which then installed cleanly with `pip`.

Finally, open an interactive shell from a **different** directory (anywhere other than `text_utility_pkg`) and confirm the import succeeds:

```
$ cd /tmp
$ python3
>>> import vsearch
>>> vsearch.search4letters('TGGACC', 'GC')
{'G', 'C'}
```

If this raises `ImportError`, the most common cause is running the shell from *inside* `text_utility_pkg` by accident, which lets Python find `vsearch.py` directly on disk rather than through the installed package — this can mask a packaging mistake that would otherwise show up immediately. Always test the import from elsewhere.

3 Task 2: Diagnostic Linting and PEP 8 Refactoring (40 minutes)

3.1 Install the Checker

```
python3 -m pip install --upgrade pycodestyle
```

3.2 Intentional Flaw Injection

Make a working copy of `vsearch.py` and deliberately introduce the following three style errors:

- (i) A missing space after a colon in a type hint (e.g., `phrase:str` instead of `phrase: str`).
- (ii) Trailing whitespace at the end of a line (spaces after the last visible character, before the newline).
- (iii) Only one blank line between two top-level function definitions, where PEP 8 requires two.

3.3 Running the Linter

```
python3 -m pycodestyle vsearch.py
```

3.4 Reading and Fixing the Output

A file containing exactly the three flaws above produces output equivalent to this (verified against a real run):

```
vsearch.py:1:26: E231 missing whitespace after ':'
vsearch.py:2:57: W291 trailing whitespace
vsearch.py:4:1: E302 expected 2 blank lines, found 0
```

Code Meaning and Fix

E231	Missing whitespace after :. Add a space after every colon in a type hint, e.g. <code>phrase: str</code> .
W291	Trailing whitespace. Delete every space or tab that appears after the last visible character on a line.
E302	Expected 2 blank lines, found fewer. Insert a second blank line between top-level function (or class) definitions.

Reading the location fields. Each line of output follows the pattern `filename:line:column:CODE message`. The E231 line above, for instance, points to line 1, column 26 — the exact character position of the offending colon — so you do not need to search the whole file by eye.

3.5 Verification

After fixing all three issues, re-run the checker:

```
python3 -m pycodestyle vsearch.py
```

A clean file produces **no output at all** and the command exits with status code 0. This absence of output *is* the passing result — unlike some tools, `pycodestyle` does not print a success banner; silence means no violations were found. You can confirm the exit code directly from the shell:

```
python3 -m pycodestyle vsearch.py; echo "Exit code: $?"
# Exit code: 0
```

4 Edge Cases and Extensions

- (i) **Re-injecting a fourth, different flaw** of your choosing (for example, a line longer than 79 characters, code E501) and confirming `pycodestyle` reports it by a code you have not already seen in this document. Look up what that code means before fixing it.
- (ii) **Reinstalling after a code change.** If you edit `vsearch.py` after installing it, the installed copy does **not** update automatically — you must rebuild (`python3 -m build --sdist`) and reinstall (`python3 -m pip install --force-reinstall dist/vsearch-1.0.tar.gz`) for the installed package to reflect your fix.
- (iii) **Version numbering.** If you bump `version='1.0'` to `version='1.1'` in `setup.py` after fixing your style errors, confirm the new tarball is named `vsearch-1.1.tar.gz`, and that installing it does not require uninstalling the old version first.

5 Suggested Time Allocation

Phase	Description	Duration
Task 1	Directory setup, <code>setup.py</code> , build, install, verify import	35 minutes
Task 2	Flaw injection, linting, fixing, re-verification	40 minutes
Documentation & Submission	Header comment, final write-up	15 minutes
Total		90 minutes

6 Deliverables

- (i) **Package Folder:** Your complete `text_utility_pkg` directory, including the final, clean `vsearch.py`, `setup.py`, `README.txt`, and the generated `dist/` folder.
- (ii) **Before/After Linter Output:** A text file or screenshot showing the `pycodestyle` output *before* your fixes (with all three injected flaws reported) and *after* (silent, exit code 0).
- (iii) **Submission Note:** A short paragraph (100–150 words) confirming that `import vsearch` succeeded from a directory other than `text_utility_pkg`, and explaining in your own words why testing the import from *inside* that folder would not have proven the package was installed correctly.

Academic Integrity

You may discuss approaches with peers during the supervised project session, but the code and documentation you submit must be written independently and must reflect your own understanding. Submissions found to be substantially copied will be treated in accordance with the institute’s academic integrity policy.